Differentiating ODDs

Syd Bauman, Helena Bermúdez Sabel, and Martin Holmes (ATOP task force)

TEI Members’ Meeting, Kraków, 2025-09-18

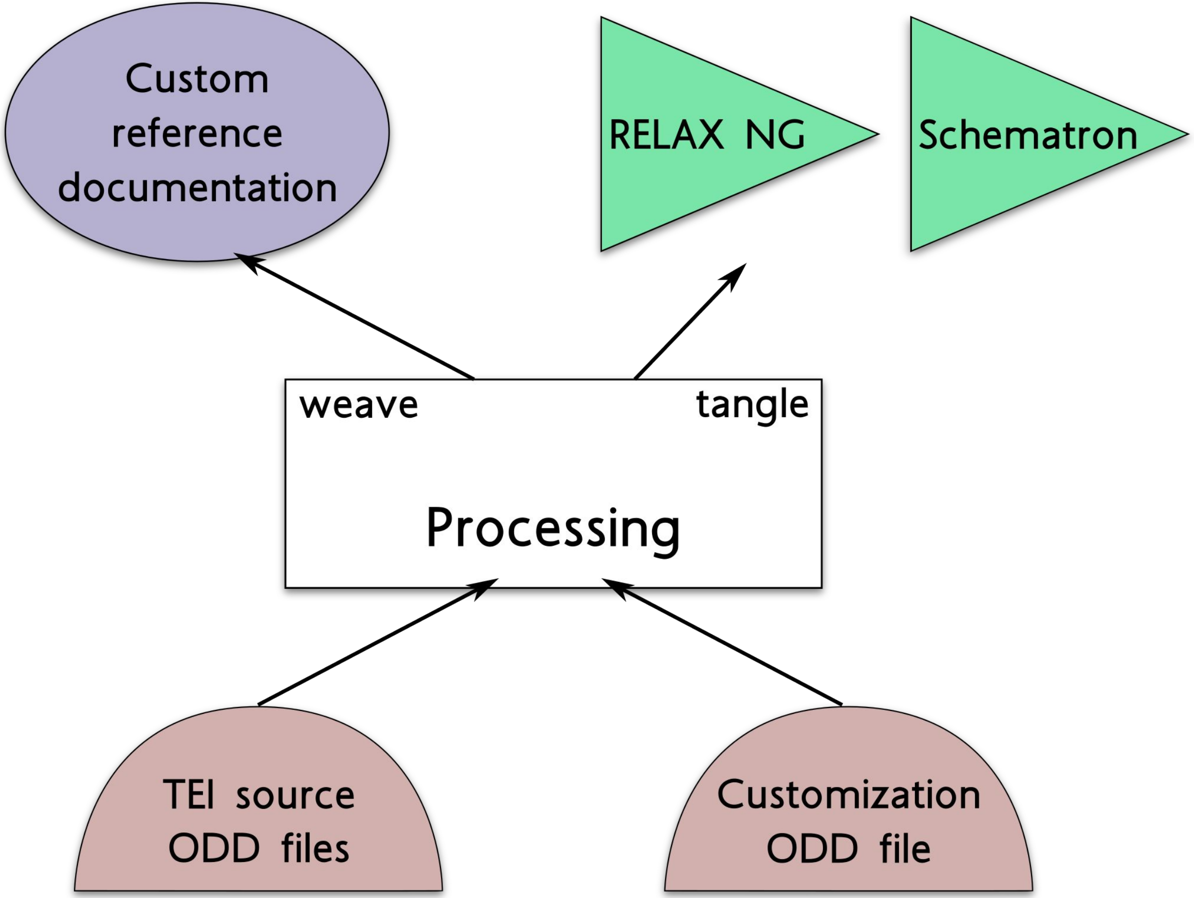
TEI

Outline

- How TEI ODD Works
- Terminology
- The original ODD conception
- Problems with that:
  - Non-TEI markup languages
  - Current Stylesheets behaviour
  - ODD chaining
- Proposed solution

TEI

How TEI ODD Works



Terminology

- **ODD**, for “one document does it all” — The TEI’s schema language that includes robust documentation and the association of output styling
- **Original base ODD** — Normally TEI P5 (e.g., either p5.xml or p5subset.xml). Contains specifications for elements, datatypes, and macros, typically organized using classes and modules
  - Also contains documentation for the system being defined, and a model for intended processing, but these do not concern us here
- **Customization ODD** — An expression of a particular schema language to be created from the specifications in its source base ODD. Typically says which parts of the base ODD are to be used (and thus which parts are to be ignored), and what additional constructs are to be added.
- **Derived ODD** — The result of a derivation process by which a customization ODD is processed against a base ODD. A derived ODD should not have any references to any base ODDs via **`@source`** on **`<*Spec>`** elements. It may be used as the base ODD for a subsequent chained derivation step. These have been called “compiled”, “expanded”, or “processed” ODDs.

Requirement and Intended Solution

ODD processing software needs to know whether an input ODD should be read as an original base ODD or as a customization ODD

The original plan (as we understand it) was that:

- A customization ODD would have a **`<schemaSpec>`** element
- An original base ODD would not

Thus software should be able to differentiate based on a simple XPath

Famous Last Words

That (simple XPath) method will not work because

- People have created stand-alone (i.e., non-TEI) markup languages using an ODD *with* a `<schemaSpec>`
  - As long as there are no `<*Ref>` elements that point at TEI, it makes no difference
- A “Compiled” ODD has a `<schemaSpec>` element, even though if & when it is used as a base ODD for chaining, it should not
- In order to chain efficiently, we would like any ODD file (whether a base or customization ODD) to be used as a base

Current Processing

```
<xsl:function name="atop:is-base-odd" as="xs:boolean">
  <xsl:param name="pOdd" as="node()"/>
  <xsl:choose>
<!--If no schemaSpec exists, it’s base (e.g. p5subset).-->
    <xsl:when test="not( $pOdd/descendant::schemaSpec )">
      <xsl:sequence select="true()"/>
    </xsl:when>
<!--If any *Ref element child of schemaSpec has @key, we can assume that it’s a customization.-->
    <xsl:when test="
        some $r in $pOdd/descendant::schemaSpec/child::*[ ends-with(local-name(), 'Ref') ]
        satisfies $r/@key">
      <xsl:sequence select="false()"/>
    </xsl:when>
<!--Any *Spec element with a mode attribute not equal to "add" means it must be a customization.-->
    <xsl:when test="
        some $s in $pOdd/descendant::schemaSpec/child::*[ ends-with(local-name(), 'Spec') ]
        satisfies $s/@mode = ('delete', 'change', 'replace')">
      <xsl:sequence select="false()"/>
    </xsl:when>
    <xsl:otherwise>
      <xsl:sequence select="true()"/>
    </xsl:otherwise>
  </xsl:choose>
</xsl:function>
```

Proposed Solution

1. Make the **`@source`** of **`<schemaSpec>`** a required attribute
    a. This means that "tei:current" is no longer a *default*, but it is still available
2. Create a special-purpose value (e.g. "**`tei:NONE`**") for use when the ODD is a base ODD

This is not a backwards-compatible change, but in the vast majority of cases the fix would be to add **`source="tei:X.Y.Z"`** and in the remaining few **`source="tei:NONE"`**, either of which is trivial to do in XSLT.

Thank you!

1. Make the `@source` of `<schemaSpec>` a required attribute
   a. This means that "tei:current" is no longer a *default*, but it is still available
2. Create a special-purpose value (e.g. `"tei:NONE"`) for use when the ODD is a base ODD

This is not a backwards-compatible change, but in the vast majority of cases the fix would be to add `source="tei:X.Y.Z"` and in the remaining few `source="tei:NONE"`, either of which is trivial to do in XSLT.

Questions?

TEI